

Pulse wave sync aliasing reduction techniques ▾

We've gone over the options for eliminating aliasing for things like square wave and sawtooth wave generation in software synthesizers. But what if you are generating for instance a pulse wave that is synced the bass guitar waveform. Is there a time domain method that could be used to smooth the corners to get the same effect, and how would you do the analysis to support the idea?

Synthesized PolyBLEP methodology for synced pulse wave anti-aliasing analysis

Time-Domain Anti-Aliasing for Synced Pulse Waves

This is a genuinely interesting variant of the problem because sync introduces a *third* class of discontinuity on top of the two you already have from the pulse wave itself, and the timing of all three is controlled by something external (the bass guitar waveform).

The Three Discontinuities

A synced pulse wave has discontinuities at:

Event	Type	Aliasing mechanism
Rising edge (pulse on)	Step (+A)	Broadband spectral splatter
Falling edge (pulse off)	Step (−A)	Same
Hard sync reset	Step + phase tear	Same, but at an <i>arbitrary</i> fractional-sample position driven by the bass waveform

The sync reset is the nastiest one — it can land anywhere between two samples and its

The sync reset is the nasty one — it can land anywhere between two samples, and its inter-sample position changes *continuously* as the bass pitch drifts, so you can't precompute it.

The Right Time-Domain Tool: PolyBLEP

BLEP (Band-Limited stEP) is the canonical time-domain answer. The idea:

The ideal bandlimited version of a unit step happening at time t_0 differs from the naive hard step by a correction term — the **BLEP residual**. Add it in after every discontinuity.

The residual is the difference between an ideal step and a sinc-based reconstruction. For real-time use, you approximate it with a **polynomial (PolyBLEP)**, applied over a window of ± 1 to ± 2 samples around each discontinuity.

The standard 2-sample quadratic PolyBLEP correction for a step at fractional position t ($0 \leq t < 1$, in units of samples):

```
// t = phase_increment normalized: how far past the discontinuity we are
// On the TRAILING side (just after):
polyblep_after(t) = t*t - 2*t + 1    →    wait, let me write this properly:

// Returns a correction to ADD to the raw sample
// t ∈ [0,1): fractional position within the current sample period
double polyblep(double t) {
    // Before discontinuity (t in [-1, 0))
    if (t < 0.0) {
        t += 1.0;
        return -t*t;
    }
    // After discontinuity (t in [0, 1))
    else {
        return t*t - 2.0*t + 1.0 - 1.0; // simplified:
    }
}

// For a step of height h at fractional sample position frac:
```

```
output[n]    += h * polyblep(frac);           // current sample correction
output[n+1] += h * polyblep(frac - 1.0); // lookahead by 1 sample
```

For higher quality, use a 4-sample cubic or the Valimaki/Pakarinen minimax polynomial.

Handling the Bass-Guitar-Driven Sync

The bass guitar gives you a *zero crossing* (or peak) event at some fractional sample position. You detect it, record the fractional phase $\tau \in [0,1)$, reset your oscillator phase, and then:

1. Apply PolyBLEP for the **sync discontinuity** using τ as the fractional position
2. If a pulse edge also falls within the same sample window (collision), apply a *second* PolyBLEP for that edge at its own τ_2
3. These can superpose — just sum the corrections

The detection of the bass waveform zero crossing itself benefits from linear interpolation between the last two samples to estimate τ :

```
// v[n-1] negative, v[n] positive → zero crossing between them
double tau = v[n-1] / (v[n-1] - v[n]); // fractional position [0,1)
```

The Analysis Framework

To demonstrate this works and quantify how well, you want a three-stage comparison:

1. Ground truth — oversampled reference

Generate the synced pulse wave at $64\times$ the target sample rate, then apply a linear-phase low-pass filter at Nyquist of the *target* rate, then downsample. This is your gold standard.

2. Naive hard sync (no correction)

Generate at target rate, apply no smoothing.

3. PolyBLEP-corrected

Same as (2) but with the corrections applied.

Metrics to compute on each:

For each signal $x[n]$ of length N :

1. FFT magnitude spectrum $\rightarrow$ identify aliasing components above Nyquist fold
2. THD+N of the fundamental period (bass sync period)
3. Alias-to-fundamental ratio (dB) for each spurious component
4. Time-domain error vs. reference: $RMSE = \sqrt{\text{mean}((x_{\text{ref}} - x_{\text{corrected}})^2)}$

What the spectrum analysis shows:

The naive pulse wave with hard sync will show mirror images of the upper harmonics folded back below Nyquist — visible as inharmonic spectral spikes that move as the bass pitch drifts. The PolyBLEP version attenuates these, with residual error proportional to $(f_s/2)^{-2k}$ where k is the polynomial order (quadratic $\rightarrow$ -40 dB/decade roll-off of alias energy above the correction window bandwidth).

Phase coherence check (specific to sync):

Since sync resets phase to an externally driven schedule, you also want to verify the correction doesn't introduce phase smear. Compute the cross-correlation between the corrected signal and the reference — a sharp peak confirms the PolyBLEP is time-aligned correctly.

Why This Works (The Physical Argument)

Any bandlimited signal reconstructed through an ideal DAC cannot contain energy above Nyquist. A hard step at a fractional sample position implicitly *assumes* the DAC outputs a staircase — which has infinite bandwidth. The PolyBLEP inserts the energy that a sinc-based reconstruction *would have put* in the samples around the transition, so the DAC output converges to the correct smooth step rather than the staircase approximation.

For the sync case specifically, the bass guitar's zero crossing naturally has limited bandwidth (it's an acoustic instrument), so the sync trigger itself is already "well-

behaved" — the aliasing comes entirely from the *response* of the pulse oscillator to that trigger, which is what the PolyBLEP corrects.

Practical Notes

- **Lookahead buffer:** PolyBLEP needs to write to $n+1$, so you need at least a 1-sample delay in your output chain (2 samples for cubic). This is almost always acceptable.
- **Sync + pitch mod interaction:** If the pulse oscillator is also being pitch-modulated, the position of the *rising/falling edge* shifts each period even without sync — both need their own τ tracking.
- **BLAMP for soft sync:** If you ever want to implement *soft sync* (where the phase is nudged rather than hard-reset), the derivative discontinuity that results needs a BLAMP (Band-Limited rAMP) correction instead — a higher-order version of the same idea applied to the corner rather than the step.

I think your vision of this is a little different from mine. We have a wild guess estimate of the frequency at this point. When we detect the "start of cycle" event from the bass guitar waveform, we emit our smoothed step, and start a timer. When the timer expires, we emit a smoothed step back to zero. Then when the next "start of cycle" event occurs, we do the same thing over. I've done no analysis, but intuition tells me that as long as the steps don't introduce aliasing, and the smoothed steps don't overlap in time, that we don't end up in too much trouble. Basically my mental model is that the discontinuous unsmoothed steps are the big problem. That's one of the reasons I asked about the analysis. Two things we can count on. The effective period of the whole waveform is within a known range, and the smoothing can be designed in such a way that the highest representable frequency could be something like 8 or 10K, well below nyquist at 48k

Reconciled user's simpler smoothed-step model with formal anti-aliasing analysis requirements

Your Model is Cleaner — Let Me Reframe Around It

Your intuition is essentially correct, and it's actually a more principled approach than PolyBLEP in this context because you're *constructing* a bandlimited signal rather than *correcting* a non-bandlimited one after the fact. The two approaches aim at the same target but from opposite directions.

Restatement of What You're Actually Building

You're constructing the pulse wave as the **superposition of two smooth transition events**:

```
output(t) = smooth_step_up(t - t_rise) + smooth_step_down(t - t_fall)
```

where `t_rise` is driven by the bass cycle detector, and `t_fall = t_rise + timer_duration`. The waveform between events is **constant** — no computation, no spectral content, no trouble.

This is correct by construction: the only spectral content the signal ever has is whatever is *in the smoothing function itself*.

The Formal Support for Your Intuition

Claim: If each smooth step is bandlimited to `f_max < f_nyquist`, and the steps don't overlap, the complete waveform is bandlimited to `f_max`.

Why:

- A constant signal (the flat tops and bottoms of the pulse) has zero spectral content
- Each transition contributes spectral content determined entirely by the shape of the smoothing kernel
- The sum of two bandlimited signals is bandlimited to the same frequency — superposition doesn't create new frequencies

- Non-overlapping means the transitions don't interact to produce a *different* shape than what you designed

The non-overlap condition is about **waveform fidelity**, not about aliasing. Overlapping smoothed steps would still be bandlimited — they'd just produce a shape you didn't intend.

Choosing and Characterizing the Smoothing Function

The smoothing kernel is everything. A few candidates and their spectral properties:

Half-cosine (raised cosine ramp) over N samples:

$$s[n] = A/2 * (1 - \cos(\pi \cdot n/N)), \quad n = 0..N-1$$
$$s[n] = A, \quad n \geq N$$

The transition occupies N samples. The spectrum of this transition rolls off at approximately $f_s / (2N)$. At 48kHz with N = 3 samples, the knee is at 8kHz. With N = 6, it's at 4kHz. You have direct, intuitive control.

Why raised cosine is a good choice: Its derivative at both endpoints is zero — it arrives at 0 and A with zero slope, meaning no *corner* at the junction with the flat regions. A linear ramp would leave a kink at each end, which is itself a (mild) spectral contributor.

The spectral roll-off argument numerically:

For a raised cosine transition of N samples at 48kHz:

N samples	Knee frequency	Energy at 10kHz	Energy at 20kHz (Nyquist)
3	~8 kHz	−3 dB	−20+ dB
6	~4 kHz	−12 dB	−40+ dB
12	~2 kHz	−24 dB	−60+ dB

For your stated target of 8-10kHz as the effective ceiling, $N = 3$ to 6 samples is all you need.

The Analysis You'd Run to Validate This

Step 1 — Characterize the kernel in isolation

Take just the N -sample transition, zero-pad it to 4096 points, compute its FFT magnitude. This gives you the "transfer function" of your smoothing. Verify it's at your target attenuation by 10kHz.

Step 2 — Generate a test waveform at known conditions

Pick a fixed bass frequency (e.g. 100 Hz) and duty cycle (50%). Generate 1 second of the smoothed pulse wave at 48kHz. This gives you a completely *periodic* signal whose Fourier series you can compute exactly and compare against the FFT.

Step 3 — Spectral analysis of the complete waveform

FFT of the 1-second waveform. You should see:

- Energy at 100 Hz, 200 Hz, 300 Hz... (the harmonic series of 100 Hz)
- Harmonic envelope shaped by the smoothing kernel's frequency response
- **Nothing above your design cutoff** except noise floor

If any energy appears at inharmonic frequencies or above the cutoff, something in the implementation is wrong (likely a phase discontinuity at the join point, meaning the flat regions aren't truly constant).

Step 4 — Sweep the bass frequency across the expected range

Now vary the bass frequency from your lowest expected value (say 40 Hz) to your highest (say 300 Hz) and verify:

- At low frequencies: plenty of room, no overlap possible, kernel is a tiny fraction of the period
- At high frequencies: compute the minimum guaranteed gap between steps


```
min_gap = min_period * duty_cycle - N_transition_samples
```

e.g. at 300 Hz, 50% duty, N=6:

```
min_gap = (48000/300) * 0.5 - 6 = 80 - 6 = 74 samples ← plenty of room
```

Step 5 — The overlap boundary condition

Find the frequency where `min_gap → 0`. That's your hard ceiling, and you can state it analytically:

```
f_max_safe = f_s / (2 * N * (1/duty_cycle))
```

Below that frequency, your design is provably safe. Above it, you'd need to shorten N, accept overlap, or clip the duty cycle.

What the Analysis Confirms

Your intuition maps to a clean statement: **you are designing a signal with bounded bandwidth, not filtering an aliased one.** The analysis just puts numbers on "how bounded" and proves the non-overlap condition holds across the operating range. The hard math is almost trivially simple — which is usually the sign that the mental model is right.

This all makes sense. I'm surprised the N is so small, but I guess it makes sense. In this model, I'm viewing the transition as a window much like for an FFT, so I can play with window shapes to get the desired tradeoff.

Validated user's insightful connection between smoothing and windowing functions

Yes, and that analogy is exact, not just approximate. The transition *is* a window function — or more precisely, it's the *integral* of a window function.

Why the Analogy is Exact

A window function `w[n]` is designed to taper a signal to zero at both ends with controlled spectral leakage. Your smoothing kernel is a **cumulative sum** (discrete integral) of a window:

$$\text{smooth_step}[n] = A * (\text{sum of } w[0..n]) / (\text{sum of } w[0..N-1])$$

The raised cosine ramp you get from the half-cosine is exactly the integral of a Hann window. The spectral properties you care about — the roll-off rate and sidelobe behavior — are inherited directly from the window's properties, just shifted by one integration (which adds one more factor of `1/f` to the roll-off, making it *steeper* than the window itself).

The Tradeoff Space You Already Know

Window → Step Integral	Roll-off	Sidelobe level	Transition width
Rectangular → Linear ramp	−20 dB/decade	High (sinc sidelobes)	Shortest
Hann → Raised cosine	−40 dB/decade	Very low	Moderate
Blackman → Blackman integral	−60 dB/decade	Extremely low	Wider
Gaussian → Error function	Smooth, no sidelobes	—	Wider still

So you're navigating exactly the same narrowness-vs-leakage tradeoff as in spectral analysis, just your "leakage" is aliasing energy and your "frequency resolution" is how sharp the transition edge is in time.

One Difference Worth Noting

-----o

In FFT windowing you apply the window to *both ends* of the analysis frame symmetrically. Here you're only using **half the window** — the rising half for the step up, the falling half for the step down. So when you look up window specifications, the N you need is half the window length cited in the literature. A "6-tap Hann window" gives you a 3-sample smooth step, which aligns with what you found earlier.

A Practical Bonus

Because the window literature is so well developed, you also get **ready-made design tables** for your synthesizer. If someone asks "can we make the cutoff 6kHz instead of 10kHz?", you just look up the window length for the desired roll-off frequency, double it for the full window spec, and you're done. No new analysis required — all that work was already done for FFT applications and transfers directly.

I'm a little clear on the window endpoints. Just to make it clear, could you provide an example of the resulting samples of a smoothed step with N of 9? I know this is a lot of filtering, but it clarifies things for me

Computed nine-sample Hann window transition values with endpoints clarified

Sure. Using a raised cosine (Hann integral) step, normalized to amplitude 1.0, with N=9 transition samples (indices 0 through 8):

s[n] = 0.5 * (1 - cos(π·n/8))

Index	Value	Visual
...	0.0000	(flat region before)
0	0.0000	
1	0.0381	

2	0.1464	
3	0.3087	
4	0.5000	
5	0.6913	
6	0.8536	
7	0.9619	
8	1.0000	
...	1.0000	(flat region after)

A few things worth noticing:

The endpoints land exactly on 0 and 1. This is the key property — the transition joins the flat regions with no discontinuity in value. And because the cosine derivative is zero at both ends, the *slope* also goes to zero at both endpoints, so there's no kink at the join either.

The shape is asymmetric in acceleration. It eases in slowly, accelerates through the middle, then eases out — exactly like a camera move. That S-curve profile is what kills the high-frequency content.

The step-down is just this table read in reverse, indices 8 down to 0. Which also means it's the same kernel, just time-flipped — consistent with treating the falling edge as the integral of the time-reversed window.

N=9 here means 9 samples including both endpoints. So the transition occupies 8 inter-sample intervals, which is the number that governs the spectral roll-off. That's the clarification on endpoints — both 0.0 and 1.0 are *within* the N samples, not outside them.

So I'm glad I did that. For $N=3$, one end is 0, the other end is 1, the middle sample is .5. It's essentially a linear ramp! That's why I was confused about the endpoints. You can't really get the tangent quality until you get a few more samples.

Verified user's insight about linear equivalence at $N=3$

Exactly right, and it's worth being precise about *why*: at $N=3$ the raised cosine and a linear ramp produce **identical samples** — 0, 0.5, 1.0. They're not just similar, they're the same sequence. The cosine curvature that gives you the zero-slope endpoints simply has no room to express itself with only one interior point. The mathematical property is there in the formula but the discrete representation can't capture it.

This has a direct spectral consequence — at $N=3$ you're getting roughly the spectral quality of a linear ramp, not a raised cosine. Which means the roll-off is closer to -20 dB/decade than the -40 dB/decade you'd expect from the Hann integral. You're not really getting the benefit you think you're paying for.

The practical upshot is that **$N=5$ is probably the real minimum** if the zero-slope endpoint behavior actually matters to you:

- $N=3$: linear ramp in disguise
- $N=5$: first point where the S-curve is genuinely visible and the endpoint tangents are meaningfully expressed
- $N=9+$: where you really get the full character of the window shape

So your earlier instinct to work through a concrete example was well placed — the small- N behavior is a genuine gotcha that the formula alone doesn't warn you about.

And looking at the table, $N=6$ gives a cutoff frequency of 4kHz. I can now see why people oversample. However, it's worth testing the actual response with such small N s, and this may still be acceptable.

Validated practical oversampling strategy and measurement considerations

Yes, and there are a couple of things working in your favor that make the real-world result potentially better than the theoretical cutoff number suggests.

The signal itself is already low in high-frequency energy. This is a bass-guitar-synced pulse wave. The fundamental and the musically meaningful harmonics are concentrated in the lower frequencies. The harmonics that would be near 4kHz are already at a high harmonic number and relatively low amplitude — so attenuating them further may be inaudible in context even if it shows up clearly on a spectrum analyzer.

The cutoff is a -3dB point, not a brick wall. The roll-off is gradual, so you're not suddenly losing everything above 4kHz — you're gently shaping the harmonic envelope in a way that might actually sound pleasing, similar to how analog filters on vintage synthesizers were never perfectly sharp either.

The oversampling insight is the key takeaway though. If you run the oscillator internally at say 4x (192kHz), $N=6$ gives you a 16kHz cutoff *before* downsampling, and after downsampling everything above 24kHz is gone anyway. You get the full benefit of the window shape with the same tiny kernel. The computational cost is modest since you're only oversampling one oscillator, not an entire mix chain.

The empirical test is still worth doing — generate the waveform, take the FFT, and look at what's actually there. The numbers might surprise you in either direction.

You keep using the word oscillator. It's not strictly an oscillator, more like an oscillator emulator. But your points are still well taken.

Fair point. The distinction matters — a true oscillator has a feedback mechanism that maintains a periodic signal autonomously. What you're describing is more of a **waveform synthesizer** or **waveform constructor** that is *driven* by an external event stream. The period isn't self-determined, it's inferred from the bass guitar detection, and the shape is assembled sample by sample from triggered events rather than continuously generated from a phase accumulator.

This actually reinforces some of what we discussed earlier. Because there's no phase accumulator drifting or being reset, the sync problem that PolyBLEP typically has to manage simply doesn't exist in your architecture. Each cycle is constructed fresh from the trigger event, and the only thing that carries over from one cycle to the next is the timer. The design is cleaner for it.

OK excellent that's what I need for now. I'll get to the implementation once I'm not distracted by other things.

Good luck with it — it's a clean design and the analysis path is straightforward when you're ready to validate it. The window shape intuition you developed will serve you well when you're tuning the tradeoff between transition sharpness and spectral cleanliness.

